

Università degli Studi di Salerno

Corso di Ingegneria del Software

ClickFly
Object Design Document
Versione 1.4



Data: 21/01/2026

Coordinatore del Progetto:

Nome	Matricola
Fabio Pennarella	0512117301

Partecipanti:

Nome	Matricola
Fabio Pennarella	0512117301
Giusy Chierchia	0512117508

Introduzione

1.1 Object Design Trade-offs

Le scelte progettuali di ClickFly sono state effettuate dando priorità ai requisiti non funzionali definiti nel RAD, in particolare Sicurezza e Affidabilità (priorità alta), seguiti da Prestazioni e Usabilità (priorità media). Scalabilità, Manutenibilità e Portabilità sono considerati obiettivi secondari.

Sicurezza vs Prestazioni

Per garantire la protezione dei dati sensibili, tutte le password sono memorizzate in forma hashata e l'accesso alle funzionalità riservate è vincolato alla gestione delle sessioni utente. Queste verifiche di autenticazione e controllo introducono un lieve overhead computazionale, accettato a fronte di un maggiore livello di sicurezza.

Affidabilità vs Complessità

Le operazioni critiche (checkout, pagamento, creazione prenotazioni) sono progettate come transazioni atomiche per evitare stati inconsistenti. L'uso di meccanismi transazionali aumenta la complessità dell'implementazione ma garantisce la coerenza dei dati.

Prestazioni vs Scalabilità futura

Il sistema è dimensionato per il carico previsto (fino a 100 utenti concorrenti). Non sono state introdotte soluzioni di distribuzione avanzata o caching complesso, poiché avrebbero aumentato la complessità senza benefici significativi.

Manutenibilità vs Tempo di sviluppo

Per rispettare i vincoli temporali del progetto, l'architettura privilegia una struttura modulare e separazione delle responsabilità, pur limitando l'utilizzo di framework o soluzioni più sofisticate che avrebbero richiesto maggiore tempo di apprendimento.

1.2 Linee guida per la documentazione dell'interfaccia

Le seguenti convenzioni sono state adottate per uniformare il design delle interfacce e garantire coerenza e manutenibilità del codice.

Naming

- Classi con nomi al singolare e descrittivi (es. `Volò`, `Prenotazione`, `Utente`).
- Metodi con frasi verbali che indicano chiaramente l'azione svolta (es. `aggiungiVolò()`, `confermaPrenotazione()`, `validaPagamento()`).
- Campi e parametri con nomi sostantivi esplicativi (es. `prezzo`, `idVolò`, `postiDisponibili`, `emailUtente`).

Gestione degli errori

- Gli errori sono segnalati tramite **eccezioni**, opportunamente gestite a livello di Service Layer.
- Le operazioni su collezioni restituiscono oggetti di tipo **List** o **Map**, garantendo robustezza rispetto a modifiche e iterazioni sugli elementi.

Conformità

- Tutte le interfacce pubbliche devono essere documentate tramite **JavaDoc**.
- Le interfacce seguono il principio di **interfacce minime**, esponendo solo i metodi strettamente necessari.

1.3 Definizioni, acronimi e abbreviazioni

- **DAO (Data Access Object)**: pattern utilizzato per separare la logica di accesso ai dati dal resto dell'applicazione.
- **DTO (Data Transfer Object)**: oggetto utilizzato per il trasferimento dei dati tra i diversi layer del sistema.

1.4 Riferimenti

- Requirements Analysis Document (RAD)
- System Design Document (SDD)
- Problem Statement Document

2. Packages

2.1 Struttura dei pacchetti

Il sistema **ClickFly** è suddiviso nei seguenti pacchetti principali:

1. Presentation Layer

- **Pacchetto:** `com.clickfly.presentazione`
- **Componenti principali:**
`VistaHome`, `VistaRicercaVoli`, `VistaCarrello`, `VistaPrenotazioni`,
`VistaLogin`

2. Service Layer

- **Pacchetto:** `com.clickfly.servizio`
- **Componenti principali:**
`ServizioAutenticazione`, `ServizioVoli`, `ServizioPrenotazioni`,
`ServizioPagamenti`

3. Persistence Layer

- **Pacchetto:** `com.clickfly.persistenza`
- **Componenti principali:**
`UtenteDAO`, `VolodAO`, `PrenotazioneDAO`, `PagamentoDAO`

Ogni pacchetto è strutturato in modo da **ridurre le dipendenze** tra i componenti e **favorire il riuso del codice**, in linea con i principi di modularità e manutenibilità.

2.2 Struttura grafica dei packages nel sistema

In questa sezione è descritta la struttura organizzativa dei file all'interno del progetto **ClickFly**.

Il progetto è organizzato come una **Java Web Application** che utilizza **Maven** come gestore delle dipendenze.

La suddivisione dei sottosistemi individuata nel **System Design Document** viene pienamente rispettata e implementata tramite i package:

- **presentazione** (Presentation Layer)
- **servizio** (Service Layer)
- **persistenza** (Persistence Layer)

All'interno di ciascun package sono realizzati i componenti definiti nel **component diagram**, rispettando le responsabilità assegnate a ogni layer.

Il progetto include inoltre la struttura standard Maven, come:

- directory **target**
- file **pom.xml**
- file di configurazione

Il tutto rispecchia l'organizzazione tipica di un progetto che utilizza le seguenti tecnologie: **Java Servlet, JSP, Bootstrap, Maven, HTML, CSS**.

Struttura dei package (ClickFly)

Il progetto ClickFly è organizzato secondo una classica architettura **Three Tier a 3 layer** (Presentation / Service / Persistence), in modo da separare chiaramente responsabilità e dipendenze.

Package **com.clickfly.modello**

Contiene le **Java Beans (entità di dominio)** che rappresentano gli oggetti definiti nel class diagram e nell'ER diagram (es. **Utente**, **Volo**, **Carrello**, **Prenotazione**, ecc.).

Queste classi incapsulano attributi e metodi base (getter/setter) e vengono usate da tutti gli altri layer.

Package **com.clickfly.presentation**

Contiene la **parte di controllo (Controller)** del pattern MVC, quindi:

- **Servlet** che ricevono richieste HTTP dal client (pagine JSP o form),
- validano input (o delegano al Service Layer),
- inoltrano la richiesta verso la vista corretta (forward/redirect),
- gestiscono sessione (**sessionId**) per utente loggato e contenuti del carrello.

Esempi tipici: servlet per **login**, **registrazione**, **ricerca voli**, **gestione carrello**, **checkout/prenotazione**, **“le mie prenotazioni”**.

Package **com.clickfly.servizio**

Contiene la **logica di business** del sistema (Service Layer).

Qui vengono implementate le regole applicative principali, ad esempio:

- autenticazione e gestione sessione utente,
- aggiunta/rimozione voli dal carrello,
- creazione prenotazione e calcolo totale,
- controllo disponibilità posti,
- gestione portafoglio/ricarica (se prevista),
- validazioni “di processo” (non solo di form).

Le servlet **non devono parlare direttamente col database**, ma chiamano i service.

Package **com.clickfly.persistenza**

Contiene l'accesso ai dati (Persistence Layer), quindi:

- **DAO Interface + DAO Implementation**
- gestione connessione database (MySQL)
- query e operazioni CRUD su tabelle: utenti, voli, carrello, prenotazioni, ecc.

Le classi DAO forniscono metodi usati dal Service Layer per leggere/scrivere informazioni persistenti.

Struttura **webapp**

Nel modulo **webapp** sono presenti tutte le risorse web del progetto:

- cartella **jsp/** con le pagine JSP (Home, Cerca voli, Login, Registrazione, Carrello, Le mie prenotazioni...)
- **META-INF/** e **WEB-INF/** per configurazioni e deployment (es. **web.xml**, eventuali configurazioni del container)
- cartella **resources/** con:
 - **images/** (logo ClickFly, icone...)
 - **styles/** (CSS, Bootstrap/custom)
 - **scripts/** (JS)
 - **templates/** (componenti riutilizzabili come **header e footer**, usati in quasi tutte le pagine)

Questa organizzazione riflette la separazione dei sottosistemi descritta nel System Design e rende il progetto più mantenibile e scalabile.

- >  database
- ▼  main
 - ▼  java
 - ▼  com.clickfly
 - >  modello
 - >  persistenza
 - >  presentazione
 - >  servizio
 - ▼  webapp
 - >  jsp
 - >  META-INF
 - ▼  resources
 - >  images
 - >  styles
 - >  templates
 - >  scripts
 - ▼  WEB-INF
 - >  test
 - >  test

Interfacce delle classi

Per ogni package verranno fornite in questa sezione le principali interfacce pubbliche e i metodi più rilevanti (Servlet → Service → DAO), descrivendo input, output ed eccezioni gestite per le funzionalità di ClickFly (registrazione, login, ricerca voli, carrello, prenotazione, gestione prenotazioni).

Interfaccia	Authentication Service
Descrizione	Authentication Service fornisce il servizio di autenticazione degli utenti, verifica dei ruoli controllo delle credenziali
Metodi	+login(Stringemail,Stringpassword):user +logout(Stringemail):void +getRuolo(Stringemail):String
Invariante di classe	-Gli utenti non autenticati non devono avere ruoli accessibili - Consistenza del ruolo

Nome Metodo	+login(Stringemail,Stringpassword):user
Descrizione	Questo metodo permette di effettuare il login, controllandole credenziali e creandola sessione per l'utente
Pre Condizioni	contextAuthenticationService::login(email:String,password:String):User pre: notemail.isEmpty()andnotpassword.isEmpty() andself.isEmailValid(email)
Post Condizioni	contextAuthenticationService::login(email:String,password:String):User post:
	result<>null implies result.email=emailand self.verifyPassword(password,result.getHashedPassword())=true and result.isAuthenticated=true and self.sessions->includes(result)
Invarianti	contextUser inv:self.hashedPassword->notEmpty()

	<code>andself.hashPassword=hash(self.hashPassword)</code>
--	---

Pre-condizioni:

- L'email e la password non devono essere vuote.
- L'email deve essere valida secondo un criterio definito nel metodo `isEmailValid`.

Post-condizioni:

- Se il risultato(`result`)non è null, significa che il login è avvenuto con successo. Inoltre:
 - L'utente restituito dal metodo deve avere l'email corrispondente a quella fornita in ingresso.
 - L'utente deve risultare autenticato. Questa condizione aggiorna lo stato dell'utente indicando che ha effettuato correttamente il login.
- L'utente autenticato deve essere aggiunto alla collezione di sessioni attive.

Invarianti:

- Ogni utente deve avere una password salvata come valore hashato e non in chiaro.

Nome Metodo	logout(String email, String password): void
Descrizione	Questo metodo consente di effettuare il logout dell'utente dal sistema ClickFly, eliminando la sessione attiva associata all'utente autenticato.
Pre Condizioni	context AuthenticationService::logout(email: String, password: String): void pre: • <code>email</code> non è vuota • <code>password</code> non è vuota • Esiste una sessione <code>s</code> tale che: ◦ <code>s.user.email = email</code> ◦ <code>s.user.isAuthenticated = true</code> • La password fornita è corretta: ◦ <code>verifyPassword(password, getUserByEmail(email).hashedPassword) = true</code>
Post Condizioni	context AuthenticationService::logout(email: String, password: String): void post: • Non esiste alcuna sessione associata all'utente con <code>email = email</code> ◦ <code>sessions->forAll(s s.user.email <> email)</code>

Invarianti	Invarianti context AuthenticationService inv: • Ogni utente autenticato deve avere una sessione attiva: ◦ <code>users->select(u u.isAuthenticated = true) ->forall(u sessions->exists(s s.user = u))</code>
------------	--

Pre-condizioni:

- L'email e la password fornite dall'utente non devono essere vuote.
- Deve esistere una sessione attiva associata all'utente identificato dall'email inserita.
- L'utente deve risultare autenticato nel sistema (`isAuthenticated = true`).
- La password fornita deve corrispondere all'hash memorizzato nel sistema per l'utente, al fine di garantire la validità dell'operazione di logout.

Post-condizioni:

- Al termine dell'esecuzione del metodo `logout`, non devono essere presenti sessioni attive associate all'utente identificato dall'email fornita.
- L'utente viene correttamente disconnesso dal sistema ClickFly.

Invarianti:

- Per ogni utente che risulta autenticato nel sistema (`isAuthenticated = true`), deve esistere una e una sola sessione attiva associata a tale utente.
- Il sistema non deve consentire la presenza di utenti autenticati privi di una sessione valida.

Nome Metodo	+getRuolo(Stringemail):String
Descrizione	Questo metodo permette di ottenere il ruolo e riconoscere che ruolo l'utente che è autenticato
Pre Condizioni	L'email e la password fornite dall'utente non devono essere vuote. Deve esistere una sessione attiva associata all'utente identificato dall'email inserita. L'utente deve risultare autenticato nel sistema (<code>isAuthenticated = true</code>). La password fornita deve corrispondere all'hash memorizzato nel sistema per l'utente, al fine di garantire la validità dell'operazione di logout.
Post Condizioni	- Al termine dell'esecuzione del metodo <code>logout</code>, non devono essere presenti sessioni attive associate all'utente identificato dall'email fornita. - L'utente viene correttamente disconnesso dal sistema ClickFly.
Invarianti	- Per ogni utente che risulta autenticato nel sistema (<code>isAuthenticated = true</code>), deve esistere una e una sola sessione attiva associata a tale utente. - Il sistema non deve consentire la presenza di utenti autenticati privi di una sessione valida.

Pre-condizioni:

- L'email fornita non deve essere vuota.
- Deve esistere una sessione attiva associata all'utente identificato dall'email inserita.
- L'utente deve risultare autenticato nel sistema ClickFly.

Post-condizioni:

- Il valore restituito (**result**) deve corrispondere al ruolo associato all'utente identificato dall'email fornita.
- Il ruolo restituito deve essere coerente con i permessi assegnati all'utente all'interno del sistema ClickFly.

Invarianti:

- Ogni utente registrato nel sistema ClickFly deve avere esattamente un ruolo assegnato.
- Il ruolo associato a un utente non deve mai essere nullo né vuoto durante il ciclo di vita dell'utente nel sistema.

Interfaccia	SecurityService
Descrizione	SecurityService fornisce i servizi per la protezione dei dati sensibili nel sistema ClickFly, in particolare per la gestione sicura delle credenziali degli utenti. Il servizio implementa meccanismi di hashing delle password e verifica delle credenziali, garantendo che nessuna password venga mai memorizzata o trasmessa in chiaro.
Metodi	hashPassword(String password) : String Restituisce l'hash sicuro della password fornita utilizzando un algoritmo di hashing robusto (es. bcrypt). verifyPassword(String insertedPassword, String hashedPassword) : boolean Verifica che la password inserita dall'utente corrisponda all'hash memorizzato nel sistema. getHashedPassword(String email) : String Restituisce la password hashata associata all'utente identificato dall'email fornita.
Invariante di classe	Ogni password nel sistema ClickFly deve essere memorizzata esclusivamente in forma hashata. L'hash restituito non deve mai essere nullo né vuoto. Nessuna operazione del sistema deve richiedere l'accesso alla password in chiaro.

Nome Metodo	+ hashPassword(String password) : String
Descrizione	Questo metodo consente di eseguire l'hashing della password inserita dall'utente all'interno del sistema ClickFly , utilizzando un algoritmo di hashing sicuro. La password non viene mai salvata né restituita in chiaro, garantendo la protezione delle credenziali dell'utente.
Pre Condizioni	context SecurityService::hashPassword(password : String) : String pre: La password fornita non deve essere vuota not password.isEmpty()
Post Condizioni	context SecurityService::hashPassword(password : String) : String post: - Il valore restituito non deve essere vuoto - Il valore restituito deve corrispondere all'hash della password fornita result <> "" and result = hash(password)
Invarianti	context SecurityService inv: - Il valore hash generato non deve mai essere vuoto - L'hash deve essere sempre coerente con la funzione di hashing

	applicata alla password <pre>not result.isEmpty() and result = hash(password)</pre>
--	--

Pre-condizioni:

- La password fornita dall'utente **non deve essere vuota**, in quanto l'algoritmo di hashing di ClickFly richiede un input valido per generare una stringa hash corretta.

Post-condizioni:

- Il risultato restituito dal metodo è **una stringa non vuota**.
- Il valore restituito rappresenta **la versione hashata della password**, generata tramite l'algoritmo di hashing sicuro adottato dal sistema.

Invarianti:

- L'output del metodo **hashPassword non può mai essere vuoto**: l'invariante garantisce che il metodo restituisca sempre una stringa valida.
- La funzione di hashing deve essere **deterministica rispetto all'input**, ovvero, a parità di password fornita, la funzione produce sempre un hash coerente secondo l'algoritmo utilizzato dal sistema ClickFly.

Nome Metodo	<pre>+ verifyPassword(String insertedPassword, String hashedPassword) : boolean</pre>
Descrizione	Questo metodo verifica se la password inserita dall'utente durante la fase di autenticazione corrisponde alla password hashata memorizzata nel sistema ClickFly. La verifica avviene applicando l'algoritmo di hashing alla password inserita e confrontando il risultato con l'hash salvato.
Pre Condizione	<pre>context SecurityService::verifyPassword(inserte dPassword : String, hashedPassword : String) : boolean pre: not insertedPassword.isEmpty() and not hashedPassword.isEmpty()</pre>
Post Condizione	<pre>context SecurityService::verifyPassword(inserte dPassword : String, hashedPassword : String) : boolean post: result = (hash(insertedPassword) = hashedPassword)</pre>
Invariante	<pre>context SecurityService::verifyPassword(inserte dPassword : String, hashedPassword : String) : boolean inv: not insertedPassword.isEmpty() and not hashedPassword.isEmpty()</pre>

	<pre>and result = (hash(insertedPassword) = hashedPassword)</pre>
--	---

Pre-condizioni:

- Assicurano che entrambe le password, quella inserita dall'utente e quella hashata memorizzata nel sistema ClickFly, non siano vuote.

Post-condizioni:

- Verificano che l'hash della password inserita corrisponda alla password hashata memorizzata nel sistema, restituendo `true` in caso di corrispondenza e `false` in caso contrario.

Invarianti:

- L'invariante garantisce che entrambe le password non siano vuote, poiché non è significativo effettuare una verifica su valori nulli o vuoti.
- Il risultato della verifica deve essere sempre coerente con l'operazione di hashing adottata dal sistema ClickFly: il valore `result` è `true` se e solo se l'hash della password inserita coincide con quello memorizzato nel sistema.

Nome Metodo	+ getHashedPassword(String email) : String
Descrizione	Questo metodo restituisce la password hashata associata a un utente del sistema ClickFly , identificato in modo univoco tramite il proprio indirizzo email.
Pre Condizioni	Questo metodo restituisce la password hashata associata a un utente del sistema ClickFly , identificato in modo univoco tramite il proprio indirizzo email.
Post Condizioni	context SecurityService::getHashedPassword(email : String) : String post: • <code>result = self.users->select(u u.email = email).hashedPassword</code>
Invarianti	context SecurityService::getHashedPassword(email : String) : String inv: • <code>not email.isEmpty()</code> • <code>self.users->exists(u u.email = email)</code> • <code>not result.isEmpty()</code> • <code>result = self.users->select(u u.email = email).hashedPassword</code>

Pre-condizioni:

- Assicura che l'email fornita non sia vuota.
- Assicura che esista un utente registrato nel sistema ClickFly associato all'email specificata.

Post-condizioni:

- Restituisce la password hashata dell'utente corrispondente all'email fornita.

Invarianti:

- L'email utilizzata per l'operazione deve sempre essere valida e riferita a un utente esistente nel sistema.
- La password restituita deve essere sempre non vuota.
- Il valore restituito deve corrispondere alla password hashata memorizzata per l'utente nel sistema ClickFly.

Interfaccia	CartService
Descrizione	CartService fornisce i servizi per la gestione del carrello voli di ClickFly. Permette all'utente di aggiungere voli al carrello, rimuoverli o modificarne la quantità di posti selezionati prima di procedere alla prenotazione.
Metodi	+ aggiungiItem(Volo v, int postiSelezionati) : boolean + rimuoviItem(Volo v) : void + modificaItem(Volo v, int postiSelezionati) : boolean
Invariante di classe	• Ogni volo presente nel carrello deve essere un volo valido e disponibile nel sistema. • Ogni volo nel carrello deve avere un numero di posti selezionati strettamente positivo. • Ogni volo può comparire una sola volta nel carrello dell'utente. • Il numero di posti selezionati per un volo non deve superare i posti disponibili per quel volo.

Nome Metodo	+ aggiungiltem(Volo v, int postiSelezionati) : boolean
Descrizione	Questo metodo permette di aggiungere un volo al carrello dell'utente. Se il volo è già presente nel carrello, il metodo aggiorna il numero di posti selezionati; in caso contrario, il volo viene aggiunto come nuovo elemento del carrello, nel rispetto della disponibilità dei posti.
Pre Condizione	context CartService::aggiungiltem(v:Volo, postiSelezionati:int) : boolean <pre> pre: v <> null and postiSelezionati > 0 and v.postiDisponibili >= postiSelezionati and v.isDisponibile = true </pre>
Post Condizione	context CartService::aggiungiltem(v:Volo, postiSelezionati:int) : boolean <pre> post: if self.carrello->exists(item item.volo = v) then item.postiSelezionati = item.postiSelezionati + postiSelezionati and item.postiSelezionati <= v.postiDisponibili and result = true else self.carrello->includes(CartItem { volo = v, postiSelezionati = postiSelezionati }) and result = true endif </pre>

Invarianti	//
------------	----

Pre-condizioni:

- Il prodotto non deve essere null e deve essere valido (disponibile nel sistema).

Post-condizioni:

- Se il prodotto già presente nel carrello, l'aggiunta deve riuscire, altrimenti l'operazione restituisce false.

Nome Metodo	+ rimuovitem(Volo v) : void
Descrizione	Questo metodo permette di rimuovere un volo dal carrello dell'utente. Se il volo non è presente nel carrello, non viene effettuata alcuna modifica.
Pre Condizioni	context CartService::rimuovitem(v:Volo) : void <pre>pre: v <> null and self.carrello->exists(item item.volo = v)</pre>
Post Condizioni	context CartService::rimuovitem(v:Volo) : void <pre>post: not self.carrello->exists(item item.volo = v)</pre>
Invarianti	

Pre-condizioni:

- Il prodotto deve essere presente nel carrello e non può essere null.

Post-condizioni:

- Una volta che il prodotto è stato rimosso, non deve più essere presente nel carrello

Nome Metodo	+ modificatem(Volo v, int posti) : boolean
Descrizione	Questo metodo permette di modificare il numero di posti selezionati per un volo presente nel carrello. Se il numero di posti è valido (positivo e compatibile con la disponibilità del volo), la modifica viene applicata; in caso contrario, l'operazione non viene eseguita.
Pre Condizione	context CartService::modificatem(v:Volo, posti: Integer) : boolean <pre>pre: v <> null and self.carrello->exists(item item.volo = v) and posti > 0 and v.postiDisponibili >= posti</pre>
Post Condizioni	context CartService::modificatem(v:Volo, posti: Integer) : boolean <pre>post: if self.carrello->exists(item item.volo = v) then item.postiSelezionati = posti and result = true else result = false</pre>
Invarianti	//

Pre-condizioni:

- Il prodotto non deve essere nullo, deve essere presente nel carrello e la quantità deve essere positiva.

Post-condizioni:

- Se la modifica della quantità è riuscita, il metodo restituisce true, altrimenti false.

Interfaccia	OrderService
Descrizione	OrderService fornisce i servizi per la gestione delle prenotazioni di voli , includendo la creazione di una prenotazione a partire dal carrello e la gestione dello stato della prenotazione durante il suo ciclo di vita.
Metodi	+ createOrder(Collection<Volo> voli, Utente u, Date dataPrenotazione, double totale) : Prenotazione + modifyStatus(Prenotazione p, String nuovoStato) : boolean + getStatus(Prenotazione p) : String
Invariante di classe	- Le prenotazioni devono avere stati validi (es. IN_ELABORAZIONE, CONFERMATA, ANNULLATA) - Le prenotazioni devono avere un totale positivo - Le prenotazioni devono essere associate a utenti validi e autenticati - Ogni prenotazione deve contenere almeno un volo

Pre-condizioni:

- La collezione di **voli** non può essere **null** né **vuota**.
- L'**utente** associato alla prenotazione deve essere **valido e autenticato** nel sistema.
- La **data della prenotazione** non può essere precedente alla data corrente.
- Il **totale** della prenotazione deve essere **maggiore di zero**.

Post-condizioni:

- Viene creata una **nuova prenotazione** contenente i dettagli forniti in ingresso.
- La prenotazione è **associata all'utente** e include **tutti i voli specificati**.
- Lo **stato iniziale** della prenotazione viene impostato correttamente (es. *IN_ELABORAZIONE*).

Nome Metodo	+modifyStatus(Prenotazione p) : boolean
Descrizione	Questo metodo consente di modificare lo stato di una prenotazione nel sistema ClickFly (ad esempio da <i>IN_ELABORAZIONE</i> a <i>CONFERMATA</i> , oppure a <i>CANCELLATA</i>), nel rispetto delle regole di transizione definite.
Pre Condizioni	context OrderService::modifyStatus(p : Prenotazione) : boolean <pre>pre: p <> null and self.prenotazioni- >includes(p) and p.stato.isModificabile()</pre>
Post Condizioni	context OrderService::modifyStatus(p : Prenotazione) : boolean <pre>post: if result = true then p.stato = p.stato.next() endif</pre>
Invarianti	//

Pre-condizioni:

- L'ordine non deve essere null.
- L'ordine deve esistere nel sistema.
- Lo stato attuale dell'ordine deve permettere modifiche.

Post-condizioni:

- Lo stato dell'ordine viene modificato, se possibile.
- Restituisce true se la modifica è stata eseguita, false altrimenti.

Nome Metodo	+getStatus(Prenotazione p) : String
Descrizione	Questo metodo restituisce lo stato attuale di una prenotazione nel sistema ClickFly (ad esempio <i>IN_ELABORAZIONE</i> , <i>CONFERMATA</i> , <i>CANCELLATA</i> , <i>COMPLETATA</i>).
Pre-Condizione	context OrderService::getStatus(p : Prenotazione) : String <pre>pre: p <> null and self.prenotazioni- >includes(p)</pre>
Post-Condizione	context OrderService::getStatus(p : Prenotazione) : String <pre>post: result = p.stato</pre>
Invariante	//

Pre-condizioni:

- L'ordine non deve essere null.
- L'ordine deve esistere nel sistema.

Post-condizioni:

- Restituisce lo stato dell'ordine specificato.

Interfaccia	Catalog Service
Descrizione	CatalogService fornisce i servizi per la gestione del catalogo dei voli disponibili su ClickFly. Consente agli operatori di sistema di creare, modificare e rimuovere voli, garantendo la coerenza dei dati e la corretta disponibilità dei posti.
Metodi	<pre> + creaVolo(idVolo : String, partenza : String, destinazione : String, data : Date, orario : Time, prezzo : Double, compagniaAerea : String, postiDisponibili : int) : Volo + modificaVolo(v : Volo) : boolean + rimuoviVolo(v : Volo) : boolean </pre>
Invariante di classe	//

Nome Metodo	<pre>+ creaVolo(String idVolo, String aeroportoPartenza, String aeroportoArrivo, Date dataPartenza, Date dataArrivo, double prezzo, String compagniaAerea, int postiDisponibili) : Volo</pre>
Descrizione	Questo metodo crea un nuovo volo nel catalogo di ClickFly utilizzando i dettagli forniti, rendendolo disponibile per la ricerca e la prenotazione.
Pre Condizione	<pre>context CatalogService::creaVolo(idVolo : String, aeroportoPartenza : String, aeroportoArrivo : String, dataPartenza : Date, dataArrivo : Date, prezzo : Double, compagniaAerea : String, postiDisponibili : Integer) : Volo</pre> pre: <pre>not idVolo.isEmpty() and not aeroportoPartenza.isEmpty() and not aeroportoArrivo.isEmpty() and not compagniaAerea.isEmpty() and prezzo > 0 and postiDisponibili >= 0 and dataPartenza >= Date::now()</pre>
Post Condizione	<pre>context CatalogService::creaVolo(idVolo : String, aeroportoPartenza : String, aeroportoArrivo : String, dataPartenza : Date, dataArrivo : Date, prezzo : Double, compagniaAerea : String, postiDisponibili : Integer) : Volo</pre>

	post: result <> null and self.catalogo->includes(result)
Invarianti	context CatalogService inv: self.catalogo->forAll(v not v.idVolo.isEmpty() and v.prezzo > 0 and v.postiDisponibili >= 0)

Pre-condizioni:

- Il prezzo deve essere maggiore di zero.
- Gli identificativi di prodotto(id,description,brand, figure)non possono essere nulli vuoti.

Post-condizioni:

- Il prodotto creato è presente nel catalogo.

Nome Modello	+ modificaVolo(Volo v) : boolean
Descrizione	Questo metodo consente di modificare i dettagli di un volo esistente nel catalogo di ClickFly, come orari, prezzo o disponibilità dei posti.
Pre Condizioni	context CatalogService::modificaVolo(v : Volo) : boolean pre: v <> null and self.catalogo->includes(v)
Post Condizioni	context CatalogService::modificaVolo(v : Volo) : boolean post: if result = true then self.catalogo->includes(v) endif
Invarianti	//

Pre-condizioni:

- Il **volo** non deve essere **null**.
- Il **volo** deve esistere nel catalogo dei voli del sistema ClickFly.

Post-condizioni:

- Il **volo** viene aggiornato con i nuovi dettagli forniti (es. orari, prezzo, disponibilità posti).
- Il metodo restituisce **true** se l'operazione di modifica va a buon fine, **false** altrimenti.

Nome Metodo	+rimuoviVolo(Volo v): boolean
Descrizione	Questo metodo rimuove un volo esistente dal catalogo dei voli del sistema ClickFly. L'operazione consente agli operatori di gestione di eliminare voli non più disponibili o non più validi.
Pre Condizione	context CatalogService::rimuoviVolo(v: Volo): boolean pre: v <> null and self.catalogoVoli->includes(v) ➤ Il volo non deve essere null. ➤ Il volo deve esistere nel catalogo dei voli del sistema.
Post Condizione	Il catalogo dei voli non può contenere voli duplicati. ➤ Ogni volo presente nel catalogo deve essere valido e correttamente identificato. ➤ La rimozione di un volo non deve compromettere la coerenza del sistema (es. prenotazioni già confermate).
Invarianti	//

Pre-condizioni:

- Il **volo** non deve essere **null**.
- Il **volo** deve esistere nel catalogo dei voli del sistema ClickFly.

Post-condizioni:

- Il volo non è più presente nel catalogo dei voli
- Il metodo restituisce true se l'operazione di rimozione è avvenuta con successo, false altrimenti.

Interfaccia	DataSource(User)
Descrizione	DataSource(User) gestisce la memorizzazione e il recupero delle informazioni relative agli utenti del sistema ClickFly, garantendo la persistenza dei dati necessari all'autenticazione, all'autorizzazione e alla gestione delle prenotazioni.
Metodi	<pre>+saveUser(User u): boolean</pre> <pre>+getUser(String email): User</pre>
Invarianti di classi	Ogni utente deve essere identificabile tramite un'email valida e univoca. Ogni utente deve possedere tutte le informazioni personali obbligatorie richieste dal sistema (es. email, password, dati anagrafici). Ogni utente deve avere un ruolo valido, conforme ai ruoli definiti nel dominio applicativo di ClickFly (es. utente registrato, operatore di gestione).

Nome Metodo	+saveUser(User u): boolean
Descrizione	Questo metodo consente di salvare un nuovo utente nel database di ClickFly , memorizzandone i dati personali, le credenziali di accesso e il ruolo associato. L'operazione viene eseguita solo se l'utente non è già presente nel sistema.
Pre Condizioni	<pre>context UserPersistence::saveUser(u: User): boolean</pre> pre: • <code>u <> null</code> • <code>not u.email.isEmpty()</code> • Non deve esistere nel sistema un altro utente con la stessa email: <pre>User.allInstances()- >forAll(existing existing.email <> u.email)</pre> • Il ruolo dell'utente deve essere valido e definito: <pre>u.role <> null</pre>
Post Condizioni	<pre>context UserPersistence::saveUser(u: User): boolean</pre> post: • Se <code>result = true</code> allora l'utente viene salvato correttamente: <pre>User.allInstances()- >includes(u)</pre>
Invarianti	Ogni utente nel sistema ClickFly deve essere identificato da un'email univoca. Ogni utente deve avere un ruolo assegnato, coerente con quelli definiti nel dominio applicativo.

	Non possono esistere due utenti con la stessa email all'interno del sistema.
--	--

Pre-condizioni:

- L'oggetto **User** non deve essere nullo.
- L'email dell'utente deve essere **valida e univoca** all'interno del sistema ClickFly.
- Il ruolo dell'utente deve essere **specificato e coerente con i ruoli previsti dal dominio applicativo di ClickFly** (ad esempio *Utente registrato* o *Gestore prenotazioni*).

Post-condizioni:

- Il metodo restituisce **true** se l'utente è stato salvato correttamente nel database di ClickFly.

Nome Metodo	+getUser(String email): User
Descrizione	Questo metodo consente di recuperare un utente registrato nel sistema ClickFly a partire dall'indirizzo email fornito, restituendo l'oggetto User associato.
Pre Condizioni	context UserPersistence::getUser(email: String): User <pre>pre: not email.isEmpty() and User.allInstances()- >exists(u u.email = email)</pre>
Post Condizioni	context UserPersistence::getUser(email: String): User <pre>post: result.email = email and result.role <> null</pre>
Invarianti	context UserPersistence <pre>inv: User.allInstances()->forAll(u not u.email.isEmpty() and u.role <> null)</pre>

Pre-condizioni:

- L'email fornita non deve essere vuota.
- Deve esistere un utente registrato nel sistema ClickFly associato all'email specificata.

Post-condizioni:

- Il metodo restituisce l'utente corrispondente all'email fornita, comprensivo delle informazioni associate all'account.

Interfaccia	DataStorage (Order) – ClickFly
Descrizione	La DataStorage(Order) gestisce la memorizzazione e il recupero dei dati relativi alle prenotazioni dei voli effettuate dagli utenti nel sistema ClickFly. Si occupa della persistenza delle informazioni associate a una prenotazione, come i voli selezionati, lo stato della prenotazione e le date rilevanti.
Metodi	+saveOrder(Order o): boolean
Invarianti di Classe	• Ogni prenotazione deve essere associata a un utente valido. • Ogni prenotazione deve contenere almeno un volo per essere considerata valida. • Ogni prenotazione deve avere uno stato valido (ad esempio: <i>in elaborazione, confermata, annullata</i>). • Le date associate alla prenotazione devono essere coerenti (la data di prenotazione non può essere successiva alla data del volo).

Nome Metodo	+saveOrder(Order o): boolean
Descrizione	Questo metodo consente di salvare una prenotazione di volo nel database del sistema ClickFly. La prenotazione viene registrata solo se è valida e non esiste già una prenotazione con lo stesso identificativo.
Pre Condizioni	context OrderPersistence::saveOrder(o: Order): boolean <pre>pre: o <> null and not o.orderId.isEmpty() and Order.allInstances()->forall(existing existing.orderId <> o.orderId) and o.user <> null and o.flights->notEmpty()</pre>
Post Condizioni	context OrderPersistence::saveOrder(o: Order): boolean <pre>post: result = true implies Order.allInstances()->includes(o)</pre> Spiegazione delle post-condizioni: • Se il metodo restituisce true, la prenotazione è stata correttamente salvata nel sistema. • La prenotazione risulta persistita nel database.
Invarianti	context OrderPersistence <pre>inv: Order.allInstances()->forall(ord ord.orderId <> null and not</pre>

	<pre>ord.orderId.isEmpty() and ord.user <> null and ord.flights->notEmpty())</pre>
--	--

Pre-condizioni

- L'oggetto **Order** (prenotazione di volo) non deve essere nullo.
- L'identificativo della prenotazione (**orderId**) deve essere univoco all'interno del sistema ClickFly, ovvero non deve esistere un'altra prenotazione con lo stesso identificativo.

Post-condizioni

- Il metodo restituisce **true** se la prenotazione è stata correttamente salvata nel database del sistema ClickFly.

Nome Metodo	+saveOrder(Order o): boolean
Descrizione	Questo metodo consente di salvare una prenotazione di volo nel database del sistema ClickFly. La prenotazione viene registrata solo se è valida e non esiste già una prenotazione con lo stesso identificativo.
Pre Condizione	context OrderPersistence::saveOrder(o: Order): boolean <pre>pre: o <> null and not o.orderId.isEmpty() and Order.allInstances()->forall(existing existing.orderId <> o.orderId) and o.user <> null and o.flights->notEmpty()</pre>
Post Condizione	context OrderPersistence::saveOrder(o: Order): boolean <pre>post: result = true implies Order.allInstances()->includes(o)</pre>
Invarianti	context OrderPersistence <pre>inv: Order.allInstances()->forall(ord ord.orderId <> null and not ord.orderId.isEmpty() and ord.user <> null and ord.flights->notEmpty())</pre>

Pre-condizioni:

- Il carrello non deve essere nullo.
- Il carrello deve essere associato un utente valido.

Post-condizioni:

- Restituisce true se il carrello è stato salvato correttamente

Interfaccia	DataStorage (Catalog)
Descrizione	La DataStorage (Catalog) gestisce la memorizzazione e il recupero dei dati relativi al catalogo dei voli disponibili nel sistema ClickFly. Include informazioni quali dettagli del volo, prezzo, disponibilità dei posti, aeroporti di partenza e arrivo, date e compagnia aerea associata.
Metodi	+saveFlight(Flight f) : boolean
Invariante di classe	Ogni volo deve avere un identificativo univoco e non vuoto. Ogni volo deve avere un prezzo maggiore di zero. Ogni volo deve essere associato a: • un aeroporto di partenza valido, • un aeroporto di arrivo valido, • una compagnia aerea esistente. Ogni volo deve specificare data e ora di partenza e di arrivo valide. La disponibilità dei posti di un volo non può mai essere negativa.

Nome Metodo	+saveFlight(Flight f) : boolean
Descrizione	Questo metodo consente di salvare un volo nel sistema ClickFly, rendendolo disponibile nel catalogo dei voli per la ricerca e la prenotazione da parte degli utenti.
Pre Condizione	context FlightsPersistence::saveFlight(f : Flight) : boolean pre: f <> null and not f.flightId.isEmpty() and f.price > 0 and f.departureAirport <> null and f.arrivalAirport <> null and f.departureDateTime > currentTime and Flight.allInstances()->forAll(existing existing.flightId <> f.flightId)
Post Condizione	context FlightsPersistence::saveFlight(f : Flight) : boolean post: result = true implies Flight.allInstances()->includes(f)
Invarianti	context FlightsPersistence inv: Flight.allInstances()->forAll(f not f.flightId.isEmpty() and f.price > 0 and f.departureAirport <> null and f.arrivalAirport <> null)

Pre-condizioni:

- L'oggetto **Flight** non deve essere nullo.
- Il volo deve avere un **identificativo univoco** (**flightId**) che lo distingue da tutti gli altri voli presenti nel sistema.
- Il volo deve avere un **prezzo maggiore di zero**, in quanto non sono ammessi voli con costo nullo o negativo.
- Il volo deve avere **aeroporto di partenza e di arrivo validi**.
- La **data e l'orario di partenza** devono essere successivi al momento corrente.

Post-condizioni:

- Il metodo restituisce **true** se il volo è stato **salvato correttamente nel sistema** ed è ora disponibile nel catalogo dei voli di ClickFly.

Interfaccia	UserView
Descrizione	serView gestisce le operazioni di interazione tra l'utente e il sistema ClickFly relative alla registrazione, autenticazione e visualizzazione dei dati dell'utente. Fornisce i servizi necessari per permettere agli utenti di creare un account, accedere al sistema e recuperare le informazioni del proprio profilo.
Metodi	+ register(String email, String username, String password1, String password2): User + getUser(String email): User
Invariante di classe	• Ogni utente deve essere identificato da un'email valida e univoca. • Le password dell'utente devono essere memorizzate esclusivamente in forma hashata. • Ogni utente deve avere un ruolo valido appartenente al dominio applicativo di ClickFly (es. User, GestoreOrdini). • Non possono esistere due utenti con la stessa email nel sistema.

Nome Metodo	+ register(String email, String username, String password1, String password2): User
Descrizione	Questo metodo permette la registrazione di un nuovo utente sulla piattaforma ClickFly, creando un account associato all'email fornita e inizializzando correttamente le credenziali di accesso.
Pre Condizioni	<pre>context UserView::register(email: String, username: String, password1: String, password2: String): User pre: not email.isEmpty() and not username.isEmpty() and password1 = password2 and User.allInstances()->forall(u u.email <> email)</pre>
Post Condizioni	<pre>context UserView::register(email: String, username: String, password1: String, password2: String): User post: result <> null and result.email = email and result.username = username and result hashedPassword = hash(password1) and result.role = 'User'</pre>
Invarianti	<pre>inv: not result.email.isEmpty() and result.role <> null and not result.hashedPassword.isEmpty()</pre>

Pre-condizioni:

- L'email non deve essere già associata a un altro account.
- Il nome utente(username)non deve essere già utilizzato.
- La password 1 deve essere uguale alla password 2.

Post-condizioni:

- Restituisce l'utente registrato se l'operazione ha successo

Nome Metodo	+ getUser(String email): User
Descrizione	Questo metodo permette di recuperare un utente registrato nel sistema ClickFly a partire dall'indirizzo email fornito.
Pre Condizione	context UserPersistence::getUser(email: String): User pre: not email.isEmpty() and User.allInstances()->exists(u u.email = email)
Post Condizione	context UserPersistence::getUser(email: String): User post: result <> null and result.email = email and result.role <> null
Invarianti	inv: not email.isEmpty() and result.email = email and result.role <> null

Spiegazioni

La pre-condizione `not email.isEmpty()` verifica che l'indirizzo email fornito non sia vuoto, poiché l'email rappresenta l'identificativo univoco dell'utente nel sistema ClickFly.

La condizione `User.allInstances()->exists(u | u.email = email)` assicura che esista almeno un utente registrato nel sistema ClickFly con l'email fornita, evitando il recupero di utenti inesistenti.

Interfaccia	CatalogView
Descrizione	CatalogView gestisce la visualizzazione del catalogo dei voli disponibili su ClickFly, fornendo supporto alla ricerca e al filtraggio dei voli in base a diversi criteri, come destinazione, compagnia aerea e prezzo. L'interfaccia consente agli utenti (guest e registrati) di consultare i voli senza modificare i dati sottostanti.
Metodi	<pre> +getFlights(): Set<Volo> +getFlightById(String id): Volo +getFlightsByAirline(String airline): Set<Volo> +getFlightsByPrice(boolean ascending): Set<Volo> +getFlightsByAirlineAndPrice(Str ing airline, boolean ascending): Set<Volo> </pre>
Invariante di classe	Il catalogo dei voli non può essere vuoto. Gli identificativi dei voli (ID volo) devono essere univoci. Il prezzo di ogni volo deve essere un valore positivo.

Nome Metodo	+getFlights(): Set<Volo>
Descrizione	Questo metodo permette di recuperare l'insieme di tutti i voli disponibili presenti nel catalogo di ClickFly, rendendoli accessibili per la visualizzazione e la ricerca da parte degli utenti.
Pre Condizioni	(Non sono richieste pre-condizioni: il metodo può essere invocato da qualsiasi utente, autenticato o guest)
Post Condizioni	context CatalogView::getFlights(): Set(Volo) post: result->notEmpty()
Invarianti	Il catalogo dei voli deve contenere solo voli validi. Ogni volo presente nel risultato deve avere: • un identificativo univoco, • un prezzo maggiore di zero, • una data di partenza valida. Il metodo restituisce sempre una collezione coerente con lo stato attuale del catalogo.

Pre-condizioni:

- Nessuna

Post-condizioni:

- Restituisce l'insieme di tutti i prodotti nel catalogo

Nome Metodo	+getFlightById(String id): Volo
Descrizione	Questo metodo permette di ottenere un volo specifico dal catalogo di ClickFly a partire dal suo identificativo univoco.
Pre Condizione	context CatalogView::getFlightById(id: String): Volo pre: not id.isEmpty() and Volo.allInstances()->exists(v v.id = id)
Post Condizione	context CatalogView::getFlightById(id: String): Volo post: result.id = id
Invarianti	L'identificativo del volo non deve essere vuoto. Ogni volo nel sistema deve avere un identificativo univoco. Il volo restituito deve appartenere al catalogo dei voli disponibili. Il metodo non modifica lo stato del sistema (operazione di sola lettura).

Pre-condizioni:

- L'ID del **volo** deve essere valido e non vuoto.
- Deve esistere nel sistema un volo associato all'ID fornito.

Post-condizioni:

- Il sistema restituisce il **volo** corrispondente all'ID specificato, se presente nel catalogo dei voli.

Nome Metodo	+getFlightsByAirline(String airline): Set<Volo>
Descrizione	Questo metodo permette di ottenere una collezione di voli associati a una determinata compagnia aerea .
Pre Condizioni	context CatalogView::getFlightsByAirline(airline: String): Set(Volo) pre: not airline.isEmpty() and Volo.allInstances()->exists(v v.compagniaAerea = airline)
Post Condizioni	context CatalogView::getFlightsByAirline(airline: String): Set(Volo) post: result->forAll(v v.compagniaAerea = airline)
Invarianti	• Ogni volo restituito deve appartenere alla compagnia aerea specificata. • Il catalogo dei voli contiene solo voli validi. • Ogni volo è identificato univocamente da un ID.

Pre-condizioni:

- Il nome della **compagnia aerea** deve essere valido e non vuoto.

Post-condizioni:

- Restituisce un insieme di **voli** appartenenti alla **compagnia aerea** specificata.

Nome Metodo	+getFlightsByPrice(boolean ascending): Set<Volo>
Descrizione	Questo metodo permette di ottenere una collezione di voli ordinati in base al prezzo , secondo il criterio specificato (crescente o decrescente).
Pre Condizione	Nessuna pre-condizione specifica. Il catalogo dei voli può essere vuoto o contenere uno o più voli.
Post Condizione	<pre>contextCatalogView::getFlightsBy Price(ascending: Boolean): Set(Volo) post: if ascending then result = Volo.allInstances()->sortedBy(v v.prezzo) else result = Volo.allInstances()->sortedBy(v v.prezzo)->reverse() endif</pre>
Invarianti	Ogni volo restituito deve avere un prezzo positivo. L'ordinamento deve rispettare il criterio indicato dal parametro ascending. Il metodo non modifica il catalogo dei voli, ma ne restituisce solo una vista ordinata.

Pre-condizioni:

- Il criterio di ordinamento per prezzo deve essere valido il parametro `ascending` deve indicare un ordinamento **crescente** (`true`) oppure **decrescente** (`false`).

Post-condizioni:

- Il metodo restituisce una collezione di **voli** ordinati in base al **prezzo del volo**.
- L'ordinamento è:
 - **crescente** se `ascending = true`;
 - **decrescente** se `ascending = false`.

Nome Metodo	+getFlightsByCompanyAndPrice(String company, boolean ascending): Set<Volo>
Descrizione	Questo metodo permette di ottenere una collezione di voli filtrati per compagnia aerea e ordinati in base al prezzo del volo .
Pre Condizione	context CatalogView::getFlightsByCompanyAndPrice(company: String, ascending: Boolean): Set(Volo) <pre>pre: not company.isEmpty() and Flight.allInstances()- >exists(f f.company = company)</pre>
Post Condizioni	context CatalogView::getFlightsByCompanyAndPrice(company: String, ascending: Boolean): Set(Volo) <pre>post: result->forAll(f f.company = company) and if ascending then result = result->sortedBy(f f.price) else result = result->sortedBy(f f.price)->reverse() endif</pre>
Invarianti	<pre>inv: result->forAll(f f.price > 0) and result->forAll(f not f.company.isEmpty())</pre>

Pre-condizioni:

- La **compagnia aerea** deve essere valida e non vuota.
- Il criterio di **ordinamento per prezzo** deve essere valido (ascendente o discendente).

Post-condizioni:

- Il metodo restituisce l'insieme dei **voli appartenenti alla compagnia aerea specificata**, ordinati per **prezzo** in ordine **crescente o decrescente** in base al valore del parametro **ascending**.

Interfaccia	OrderView
Descrizione	OrderView gestisce la visualizzazione delle prenotazioni dei voli effettuate dagli utenti e consente agli operatori di gestione di monitorare e aggiornare lo stato delle prenotazioni presenti nel sistema.
Metodi	<code>+getOrderById(String orderId) : Order</code> <code>+getOrdersByUser(String email) : Set<Order></code> <code>+getAllOrders() : Set<Order></code>
Invariante di classi	L'ID della prenotazione deve essere univoco e non nullo. Ogni prenotazione deve essere associata a un utente valido. Ogni prenotazione deve essere associata ad almeno un volo valido.

Nome Metodo	+getOrderById(String orderId) : Order
Descrizione	Questo metodo permette di ottenere una prenotazione di volo a partire dal suo identificativo univoco .
Pre Condizione	context OrderView::getOrderById(orderId : String) : Order <pre>pre: not orderId.isEmpty() and Order.allInstances()- >exists(o o.id = orderId)</pre>
Post Condizione	context OrderView::getOrderById(orderId : String) : Order <pre>post: result.id = orderId</pre>
Invarianti	Ogni prenotazione deve avere un identificativo univoco e non nullo. La prenotazione restituita deve essere associata a un utente valido. La prenotazione deve riferirsi a un volo esistente nel sistem

Pre-condizioni:

- L'**ID della prenotazione** deve essere valido e non vuoto.
- Deve esistere nel sistema **una prenotazione associata all'ID fornito**.

❖ Post-condizioni:

- Il sistema restituisce la **prenotazione di volo** corrispondente all'ID specificato, se presente.

Nome Metodo	<code>+getOrdersByUser(String email): Set<Order></code>
Descrizione	Questo metodo permette di ottenere l'insieme delle prenotazioni di voli associate a un determinato utente, identificato tramite l'indirizzo email.
Pre Condizioni	context OrderView::getOrdersByUser(email: String): Set(Order) <pre>pre: not email.isEmpty() and User.allInstances()- >exists(u u.email = email)</pre>
Post Condizioni	context OrderView::getOrdersByUser(email: String): Set(Order) <pre>post: result->forAll(o o.user.email = email)</pre>
Invarianti	Ogni prenotazione restituita deve essere associata a un utente valido. Le prenotazioni visualizzate devono appartenere esclusivamente all'utente autenticato. L'email dell'utente associato a ciascuna prenotazione non deve essere vuota.

Pre-condizioni:

- L'email fornita deve essere valida e non vuota.
- Deve esistere un utente registrato nel sistema ClickFly associato all'email specificata.

Post-condizioni:

- Il metodo restituisce un insieme di **prenotazioni di voli** associate all'utente identificato dall'email fornita.
- Tutti gli ordini restituiti devono appartenere esclusivamente all'utente specificato.

Nome Metodo	+getAllOrders(): Set<Order>
Descrizione	Questo metodo permette di recuperare tutte le prenotazioni di voli presenti nel sistema ClickFly. È destinato alle figure di gestione degli ordini e consente la visualizzazione globale delle prenotazioni registrate.
Pre Condizioni	<i>Nessuna pre-condizione esplicita.</i> Il metodo è invocabile solo da componenti autorizzati (es. gestore ordini), come stabilito dal controllo degli accessi a livello di sistema.
Post Condizioni	context OrderView::getAllOrders(): Set(Order) <i>post:</i> <i>result->notEmpty()</i> ➤ Il risultato contiene l'insieme di tutte le prenotazioni registrate nel sistema ClickFly.
Invarianti	Ogni ordine restituito deve avere un identificativo univoco. ➤ Ogni ordine deve essere associato a un utente valido. ➤ Ogni ordine deve avere uno stato valido (es. "in elaborazione", "confermato", "annullato").

Nome Metodo	+getAllOrders(): Set<Order>
Descrizione	Questo metodo permette di recuperare tutte le prenotazioni di voli presenti nel sistema ClickFly . È destinato alle figure di gestione degli ordini e consente la visualizzazione globale delle prenotazioni registrate.
Pre Condizione	<i>Nessuna pre-condizione esplicita.</i> Il metodo è invocabile solo da componenti autorizzati (es. gestore ordini), come stabilito dal controllo degli accessi a livello di sistema.
Post Condizione	context OrderView::getAllOrders(): Set(Order) <i>post:</i> <i>result->notEmpty()</i> Il risultato contiene l'insieme di tutte le prenotazioni registrate nel sistema ClickFly.
Invariante	Ogni ordine restituito deve avere un identificativo univoco . Ogni ordine deve essere associato a un utente valido . Ogni ordine deve avere uno stato valido (es. "in elaborazione", "confermato", "annullato").

Pre-condizioni:

- Non è richiesta alcuna pre-condizione specifica per l'esecuzione del metodo.
- L'accesso al metodo è regolato dal sistema di controllo degli accessi, che ne consente l'utilizzo esclusivamente agli utenti autorizzati (es. gestore ordini).

Post-condizioni:

- Il metodo restituisce l'insieme di **tutte le prenotazioni di voli presenti nel sistema ClickFly**.
- Ogni ordine restituito contiene informazioni coerenti e valide, come utente associato, voli prenotati, stato e totale.

Interfaccia	CartView
Descrizione	CartView gestisce la visualizzazione e l'accesso ai dati del carrello voli associato a un utente del sistema ClickFly. Consente all'utente autenticato di visualizzare i voli selezionati prima della conferma della prenotazione.
Metodi	+ getCartByUser(String email) : Cart
Invariante di classe	Ogni carrello deve essere associato a un utente valido e autenticato. Un carrello non può esistere senza un utente proprietario. Ogni carrello contiene esclusivamente voli selezionati dall'utente associato.

Nome Metodo	+ getCartByUser(String email) : Cart
Descrizione	Questo metodo permette di recuperare il carrello voli associato a un utente , identificato tramite la sua email. Il carrello contiene i voli selezionati dall'utente prima della conferma della prenotazione.
Pre Condizione	context CartView::getCartByUser(email: String) : Cart pre: not email.isEmpty() and User.allInstances()->exists(u u.email = email) L'email fornita non deve essere vuota. Deve esistere un utente registrato nel sistema con l'email specificata.
Post Condizione	context CartView::getCartByUser(email: String) : Cart post: result <> null and result.user.email = email Il metodo restituisce un carrello valido. Il carrello restituito è associato all'utente identificato dall'email fornita.
Invariante	inv: result.user.email = email and result.user <> null Ogni carrello è sempre associato a un utente valido. Non può esistere un carrello senza un utente proprietario.

Pre-condizioni:

- L'email fornita deve essere valida e non vuota.
- Deve esistere un utente registrato nel sistema ClickFly associato all'email specificata.

Post-condizioni:

- Il metodo restituisce il carrello voli associato all'utente identificato dall'email fornita, contenente le eventuali prenotazioni selezionate ma non ancora confermate.

Design Patterns

Per garantire una progettazione **robusta, modulare, scalabile e facilmente manutenibile**, nel sistema **ClickFly** sono stati adottati i seguenti design pattern architetturali e strutturali.

4.1 Three Tier

Motivo

Il pattern Three Tier è stato adottato per separare in modo chiaro:

- la **logica di presentazione**,
- la **logica applicativa e di controllo**,
- la **gestione dei dati e delle entità di dominio**.

Questa separazione consente una maggiore manutenibilità del codice, facilita l'evoluzione del sistema e riduce le dipendenze tra componenti.

Applicazione in ClickFly

- **Persistence Layer**

Include le classi che rappresentano il dominio applicativo e l'accesso ai dati, tra cui:

- **Voło**
- **Prenotazione**
- **Carrello**
- **Utente**
- **DAO** (**VołoDAO**, **PrenotazioneDAO**, **UtenteDAO**)

- **Presentation Layer**

Comprende le pagine JSP e le classi di visualizzazione responsabili dell'interazione con l'utente, ad esempio:

- **CatalogView** (visualizzazione e ricerca voli)
- **CartView** (visualizzazione carrello)
- **OrderView** (visualizzazione prenotazioni)

- `UserView` (registrazione, login, profilo)
- **Application Layer**
Implementato tramite Servlet e Service Layer, gestisce le richieste HTTP e coordina la logica applicativa:
 - `AuthenticationService`
 - `CatalogService`
 - `CartService`
 - `OrderService`

4.2 DAO (Data Access Object)

Motivo

Il pattern DAO è stato utilizzato per separare la **logica di accesso ai dati** dalla **logica di business**, rendendo il sistema indipendente dal DBMS utilizzato e facilitando eventuali modifiche future alla persistenza.

Applicazione in ClickFly

- Le classi DAO incapsulano tutte le operazioni di lettura e scrittura sul database MySQL, tra cui:
 - `VoioDAO`
 - `PrenotazioneDAO`
 - `UtenteDAO`
 - `CarrelloDAO`
- Ogni DAO fornisce metodi specifici per:
 - recuperare voli disponibili,
 - salvare e aggiornare prenotazioni,
 - gestire utenti e sessioni,
 - garantire coerenza e integrità dei dati.

Glossario

Autenticazione: Processo di verifica dell'identità di un utente tramite credenziali, come email e password, per consentire l'accesso alle funzionalità del sistema ClickFly.

Autorizzazione: Processo che verifica se un utente autenticato possiede i permessi necessari per eseguire una determinata azione (es. prenotare un volo, modificare una prenotazione).

Cache locale: Memorizzazione temporanea di dati (es. risultati di ricerca voli o dati di sessione) vicino al punto di utilizzo per migliorare le prestazioni del sistema.

Caching: Tecnica che consente di memorizzare temporaneamente dati frequentemente utilizzati, riducendo l'accesso al database e migliorando i tempi di risposta.

Carrello: Struttura temporanea che contiene i voli selezionati da un utente prima della conferma della prenotazione.

Collezione: Struttura dati che memorizza più elementi (es. liste, set), utilizzata per gestire voli, prenotazioni o risultati di ricerca.

Configurazione: Insieme di impostazioni e parametri che definiscono il comportamento del sistema ClickFly (es. connessione al database, gestione sessioni).

Data Access Object (DAO): Pattern architetturale che separa la logica di accesso ai dati dalla logica applicativa, utilizzato per la gestione di utenti, voli e prenotazioni.

Data Transfer Object (DTO): Oggetto utilizzato per trasferire dati tra i diversi layer del sistema (Presentation, Service, Persistence).

Eccezione di runtime: Errore che si verifica durante l'esecuzione del programma (es. errore di pagamento o indisponibilità posti) e che richiede una gestione immediata.

Eccezioni: Meccanismo utilizzato per intercettare e gestire errori o comportamenti inattesi durante l'esecuzione del codice.

Framework: Struttura software che fornisce funzionalità generali per semplificare lo sviluppo dell'applicazione (es. Bootstrap per il front-end).

Gestione errori: Insieme di strategie adottate per intercettare, segnalare e gestire situazioni anomale, garantendo la stabilità del sistema.

Hashing: Processo di conversione di dati sensibili (come le password) in una stringa univoca e non reversibile, utilizzato per garantire la sicurezza.

Invarianti: Condizioni che devono essere sempre vere durante il ciclo di vita di un oggetto o di una classe (es. una prenotazione deve essere associata a un utente valido).

Interfaccia: Contratto che definisce i metodi pubblici e il comportamento di una classe o di un servizio.

Layer: Livello logico dell'architettura software che separa responsabilità specifiche (Presentation Layer, Service Layer, Persistence Layer).

Metodo: Funzione associata a un oggetto o a una classe che può essere invocata per eseguire una specifica operazione.

Model-View-Controller (MVC): Pattern architetturale che separa la logica di presentazione (View), la logica applicativa (Controller) e la gestione dei dati (Model).

Pattern di progettazione: Soluzione generica e riutilizzabile per problemi ricorrenti nella progettazione del software.

Post-condizioni: Condizioni che devono essere vere dopo l'esecuzione di un metodo o di una funzione.

Pre-condizioni: Condizioni che devono essere vere prima dell'esecuzione di un metodo o di una funzione.

Prenotazione: Entità che rappresenta l'acquisto confermato di uno o più posti su un volo da parte di un utente.

Query: Richiesta di informazioni o operazioni effettuata su un database (es. ricerca voli disponibili).

Ruoli: Permessi o responsabilità assegnati agli utenti del sistema (es. utente registrato, gestore ordini).

Sessione: Contesto attivo che rappresenta un utente autenticato e le sue interazioni con il sistema ClickFly.

Set: Struttura dati che memorizza un insieme di elementi unici, senza duplicati (es. insieme di voli).

Sicurezza: Insieme di misure e tecniche adottate per proteggere dati sensibili e accessi da utilizzi non autorizzati.

Validazione: Processo che verifica che i dati forniti dall'utente rispettino criteri e requisiti definiti dal sistema (es. formato email, disponibilità posti).